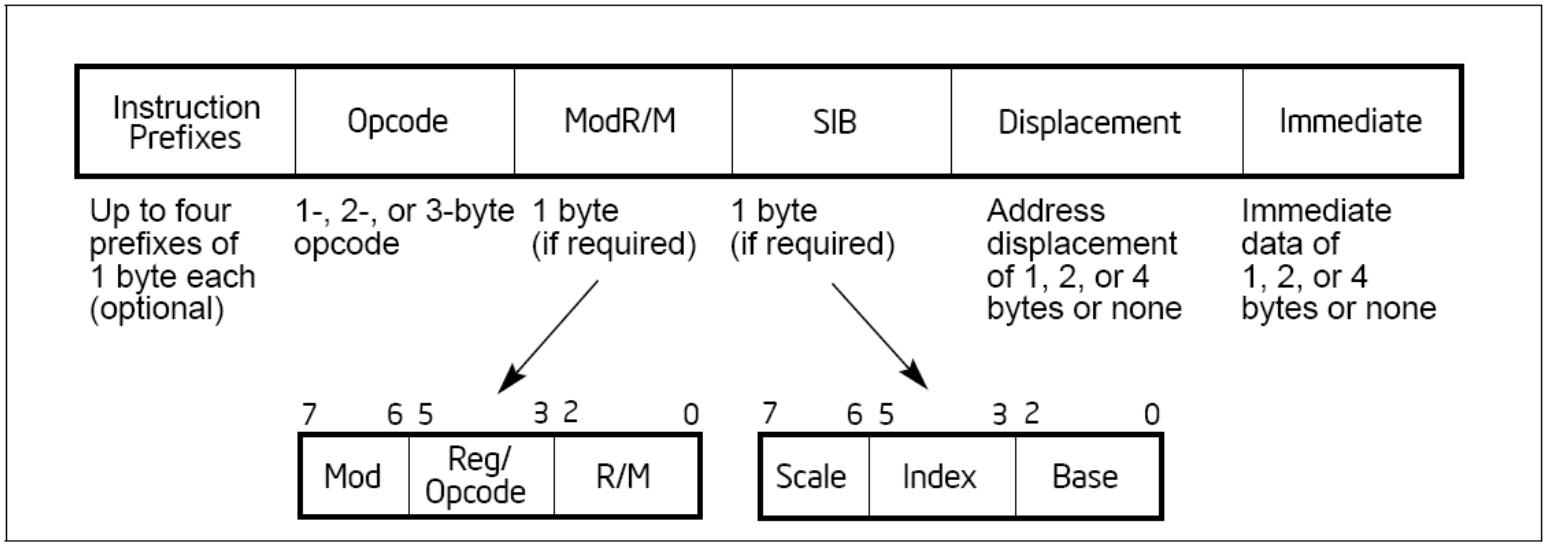
ISA 的权衡：统一解码（Uniform Decode）

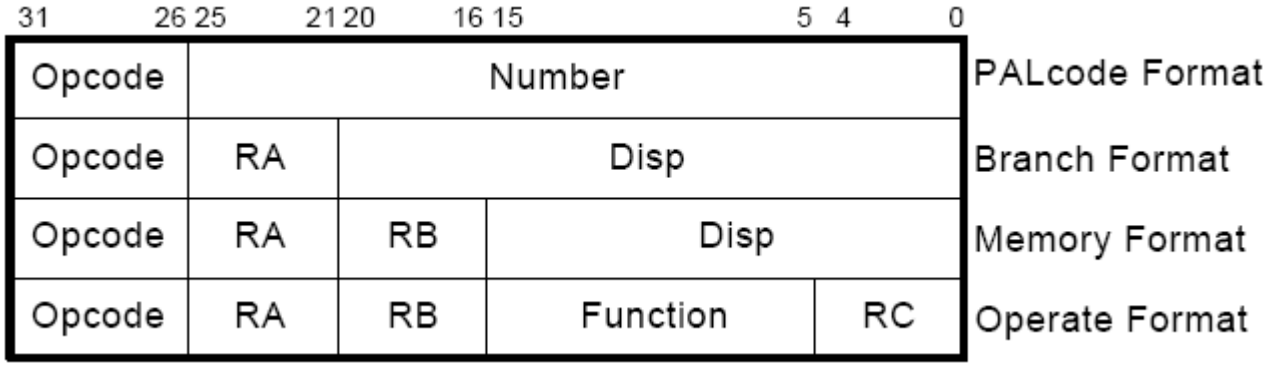
- **统一解码（Uniform Decode）定义：指令中相同的位对应相同的含义。**
  - 操作码（Opcode）始终位于相同位置。
  - 操作数说明符、立即数等也是如此。
  - 如 Alpha、MIPS、SPARC 等许多 **RISC** 架构。
  - **优点：**
    - 解码较为简单，硬件设计简化。
    - 支持并行性：可在分支指令知道之前生成目标地址。
  - **缺点：**
    - 限制了指令格式（可能导致指令数减少？）或者浪费空间。
- **非统一解码（Non-uniform Decode）**
  - 例如：x86 中，操作码（Opcode）可以是第 1 到第 7 字节。
    - **优点：**更加紧凑和强大的指令格式。
    - **缺点：**更加复杂的指令解码逻辑。

x86 与 Alpha 指令格式的对比

- x86 指令格式

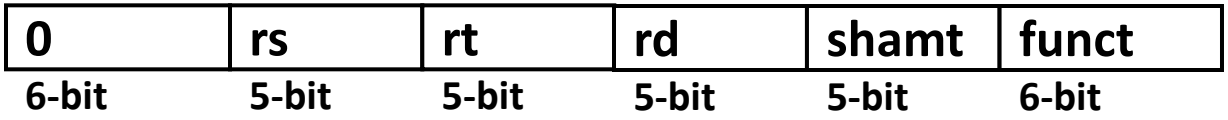


- Alpha 指令格式



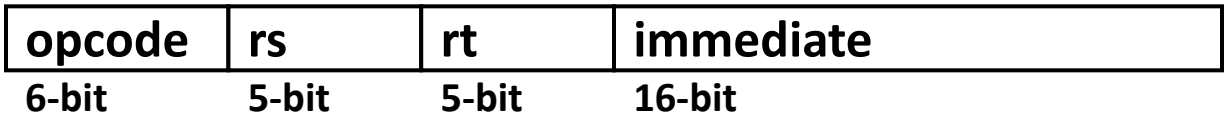
指令

- R 类型指令（R-type），3 个寄存器操作数



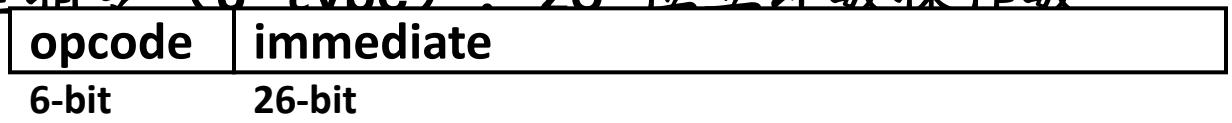
R-type

- I 类型指令（I-type），2 个寄存器操作数和 16 位立即数操作数



I-type

- J 类型指令（J-type），26 位立即数操作数



J-type

- 简易解码（Simple Decoding）
  - 每条指令长度为 4 字节，无论指令格式如何
  - 必须为 4 字节对齐（PC 的最低 2 位必须为 00）
  - 格式和字段容易在硬件中提取

ARM指令

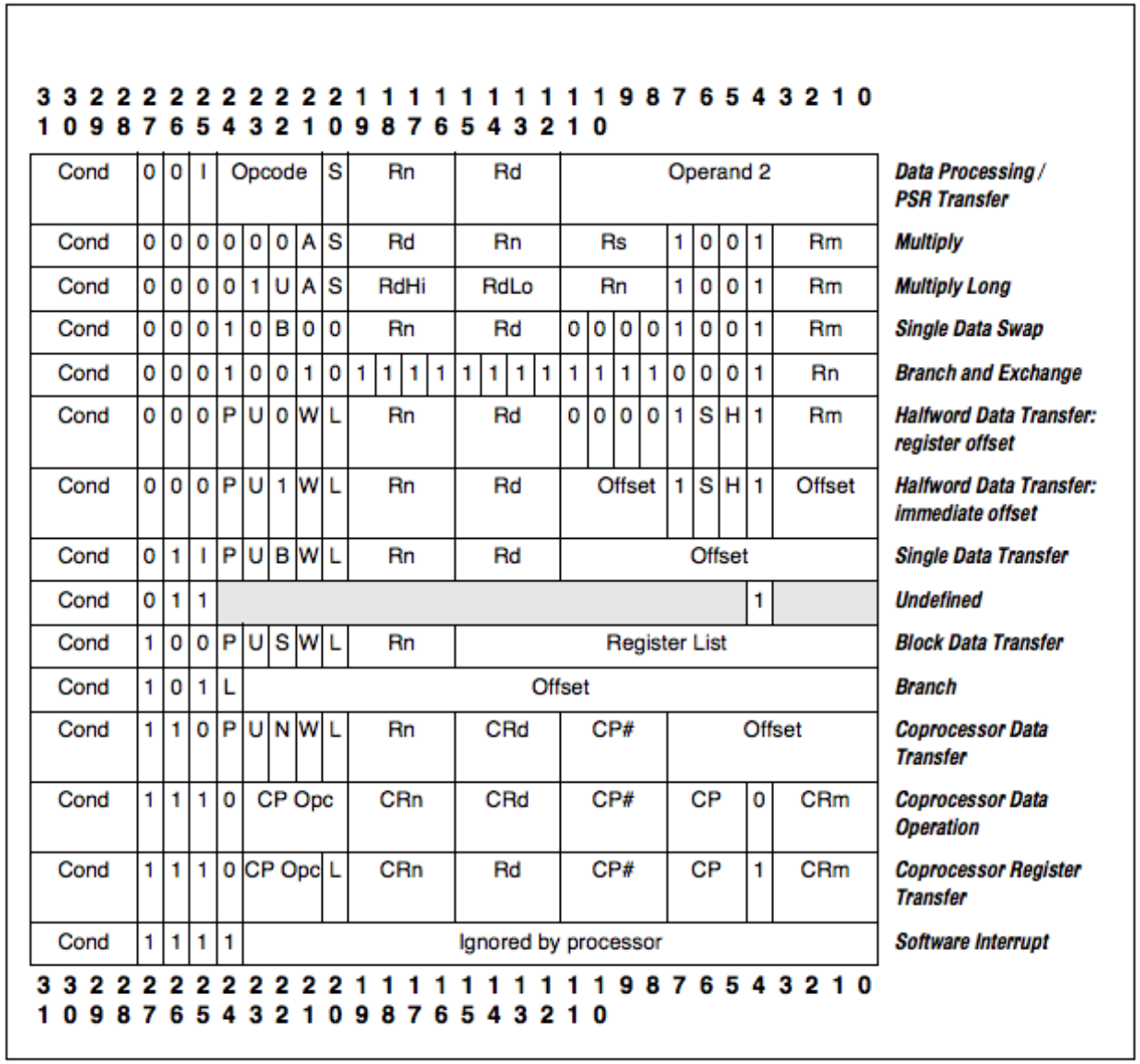


Figure 4-1: ARM instruction set formats

关于长度与统一性(Length and Uniformity)

- **统一解码通常与固定长度指令配合使用**
  - 固定长度的指令格式，使得每条指令的解码过程相对简洁。

- **在 可变长度指令集（ISA） 中，统一解码通常是指相同长度指令的特性。**
  - 很难将统一解码作为不同长度指令的共同特性。

关于 RISC 与 CISC 的小结

- 通常，，，，

- **RISC（精简指令集计算机）**
  - 简单指令
  - 固定长度
  - 统一解码
  - 较少的寻址模式

- **CISC（复杂指令集计算机）**
  - 复杂指令
  - 可变长度
  - 非统一解码
  - 较多的寻址模式

ISA 的权衡：寄存器数量

- **影响因素：**
  - 寄存器地址编码所用的位数
  - 存储在快速存储器（寄存器文件）中的数值数量
  - 寄存器文件的大小、访问时间和功耗
- **大量寄存器的优势：**
  - 允许编译器进行更好的寄存器分配（并优化程序），减少寄存器保存与恢复的次数。
  - 缺点：
    - 增加指令大小。
    - 增加寄存器文件的大小。

ISA 的权衡：寻址模式

- **寻址模式的定义：**
  - 寻址模式决定了如何获取操作数。
    - **寄存器寻址（Register）**
    - **立即数寻址（Immediate）**
    - **内存寻址（Memory）：**如偏移量、寄存器间接、索引、自增、自减等。
- **更多寻址模式的优势：**
  - 帮助更好地支持编程构造（如数组、基于指针的访问）。
  - 缺点：
    - 增加了架构设计的复杂度。
    - 对编译器的要求更高，增加了编译器的设计难度。
  - 参考文献：
    Wulf, "Compilers and Computer Architecture," IEEE Computer 1981

立即寻址（Immediate Addressing）

- 在立即寻址模式中，操作数是指令中直接给出的常数值（立即数）。
- 计算方式：
  - 立即数是指令中直接提供的数值。
- 示例：
  - `addi $t0, $t1, 10`
- 这条指令将寄存器 $t1 的值加上立即数 10，并将结果存储到 $t0 中。

寄存器寻址（Register Addressing）

- 在寄存器寻址模式中，操作数存储在寄存器中。
- 计算方式：
  - 操作数直接来自寄存器。
- 示例：
  - `add $t0, $t1, $t2`
- 这条指令将 $t1 和 $t2 寄存器的值相加，并将结果存储到 $t0 中。

基址寻址（Base Addressing）

- 基址寻址使用一个寄存器作为基地址，再加上一个偏移量来计算实际地址。
- 计算方式：
  - 有效地址 = 基地址寄存器 + 偏移量
- 示例：
  - `lw $t0, 4($t1)`
- 这条指令将寄存器 $t1 的值作为基地址，再加上偏移量 4，得到内存地址，然后从该地址加载数据到寄存器 $t0 中。
- 计算：假设 $t1 的值为 0x1000，则：
  - 有效地址 = 0x1000 + 4 = 0x1004
  - 从内存地址 0x1004 读取数据。

偏移量寻址（Offset Addressing）

- 偏移量寻址类似于基址寻址，但偏移量通常是一个常数。
- 计算方式：
  - 有效地址 = 寄存器值 + 偏移量
- 示例：
  - `sw $t0, 8($sp)`
- 这条指令将寄存器 $t0 的值存储到内存中，存储位置为寄存器 $sp 的值加上偏移量 8。
- 计算：假设 $sp 的值为 0x2000，则：
  - 有效地址 = 0x2000 + 8 = 0x2008
  - 将 $t0 的值存储到内存地址 0x2008。

PC相对寻址（PC-relative Addressing）

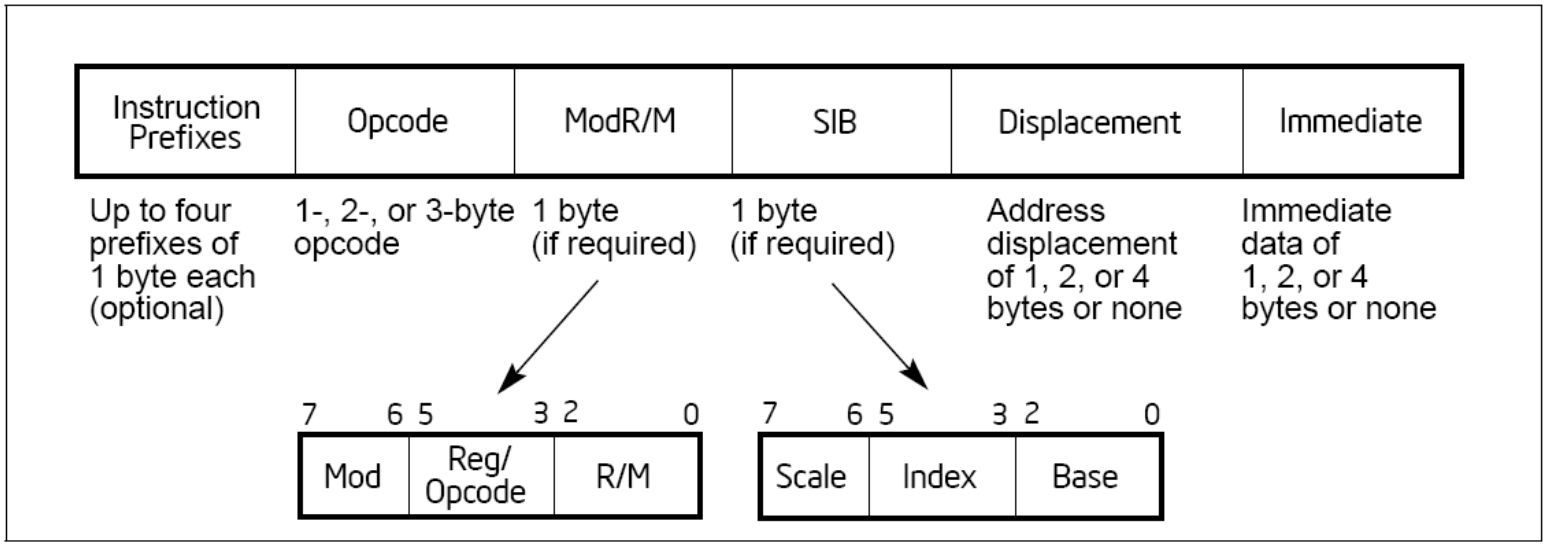
- 相对寻址模式使用当前程序计数器（PC）的值和指令中的偏移量来计算有效地址。
- 计算方式：
  - 有效地址 = 当前PC值 + 偏移量
    偏移量是指令中给定的常数，通常是一个字（4 字节）为单位的偏移。
- 示例：
  - beq $t0, $t1, label
- 这条指令检查 $t0 和 $t1 是否相等，如果相等，则跳转到标签 label 所在的地址。
- 计算：假设当前 PC 值是 0x1004，指令中的偏移量为 -4（相对于当前地址的偏移）。
  - 有效地址 = 0x1004 + (-4) = 0x1000
  - 如果 $t0 和 $t1 相等，则跳转到地址 0x1000。

寄存器间接寻址（Indirect Addressing）

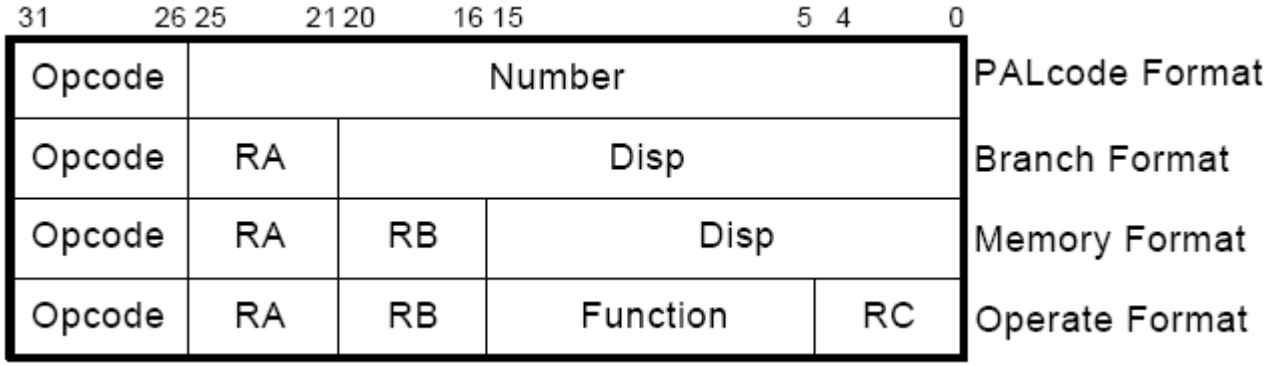
- 间接寻址模式通过寄存器存储一个地址，然后通过该地址访问数据。
- 计算方式：
  - 有效地址 = 寄存器值中的内容
    寄存器存储的是一个指针，指向实际的操作数。
- 示例：
  - `lw $t0, 0($t1)`
- 这条指令将寄存器 $t1 中存储的地址作为指向内存的指针，从该地址加载数据到 $t0 中。
- 计算：
  - 假设 $t1 中的值为 0x3000，从内存地址 0x3000 读取数据。

x86 与 Alpha 指令格式的对比

- x86 指令格式



- Alpha 指令格式



X86 SIB-D寻址模式

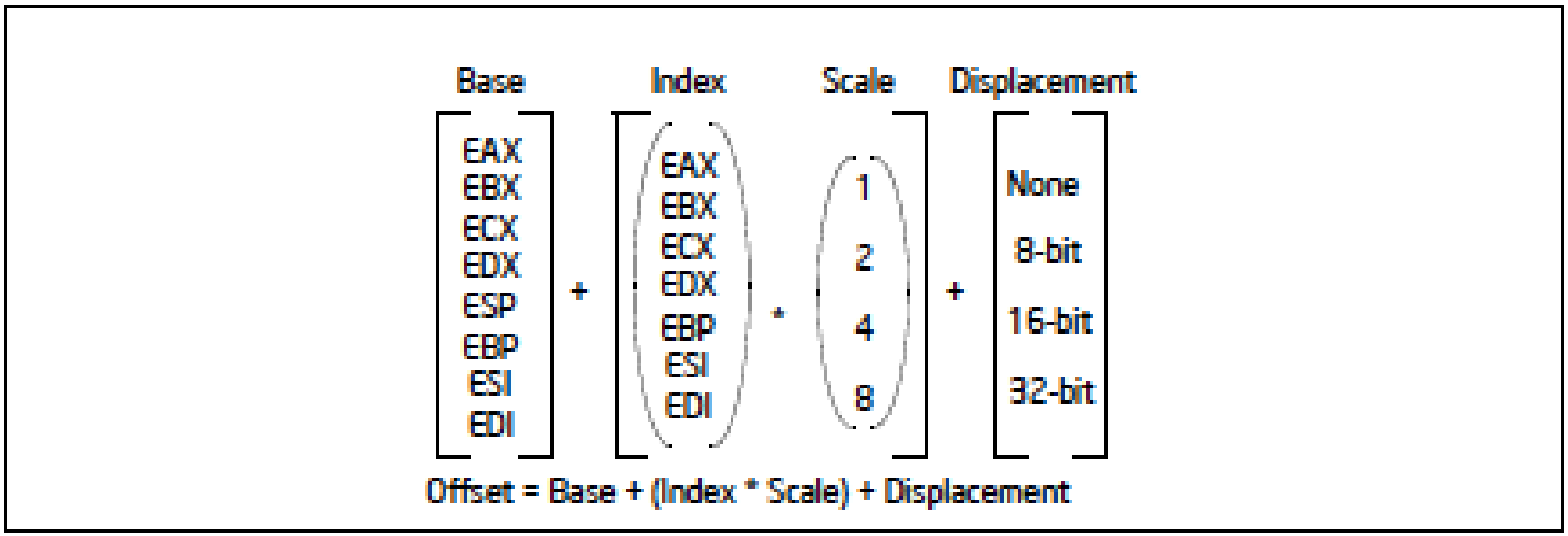
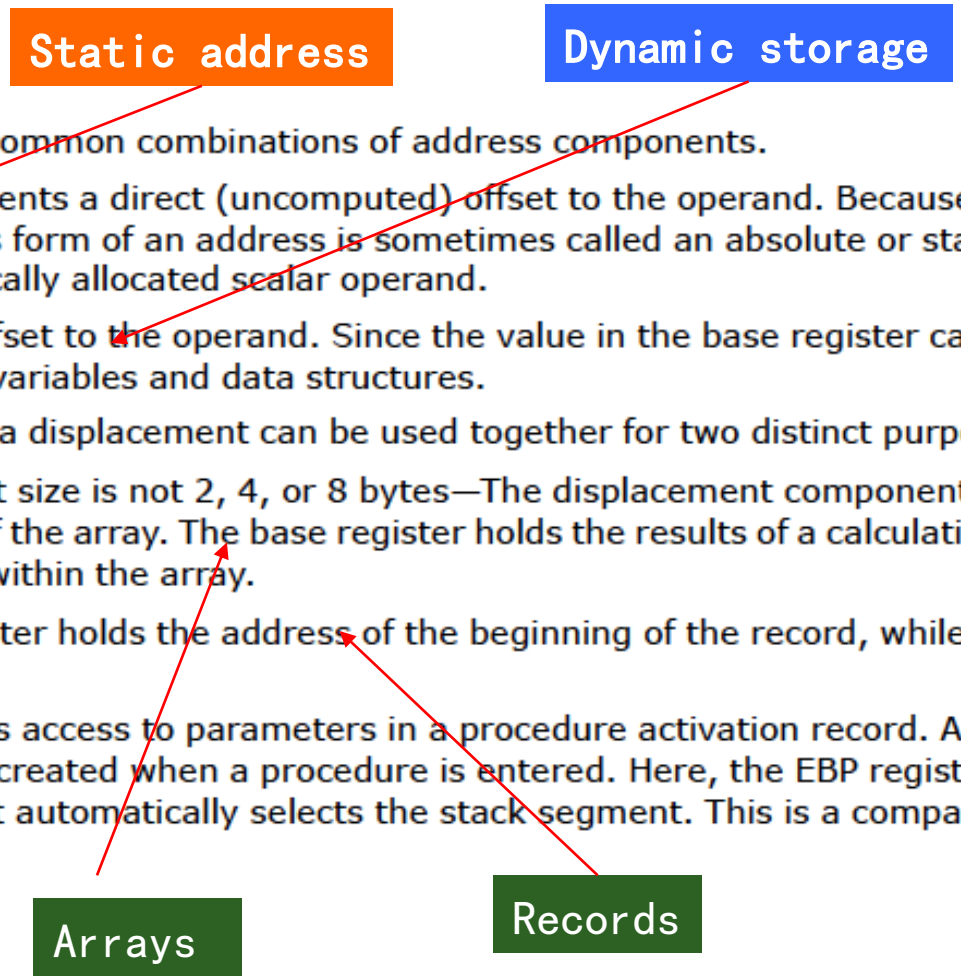


Figure 3-11. Offset (or Effective Address) Computation

x86 Manual Vol. 1, page 3-22

X86手册：寻址模式的建议用法



The following addressing modes suggest uses for common combinations of address components.

- **Displacement** — A displacement alone represents a direct (uncomputed) offset to the operand. Because the displacement is encoded in the instruction, this form of an address is sometimes called an absolute or static address. It is commonly used to access a statically allocated scalar operand.
- **Base** — A base alone represents an indirect offset to the operand. Since the value in the base register can change, it can be used for dynamic storage of variables and data structures.
- **Base + Displacement** — A base register and a displacement can be used together for two distinct purposes:
  - As an index into an array when the element size is not 2, 4, or 8 bytes—The displacement component encodes the static offset to the beginning of the array. The base register holds the results of a calculation to determine the offset to a specific element within the array.
  - To access a field of a record: the base register holds the address of the beginning of the record, while the displacement is a static offset to the field.

  An important special case of this combination is access to parameters in a procedure activation record. A procedure activation record is the stack frame created when a procedure is entered. Here, the EBP register is the best choice for the base register, because it automatically selects the stack segment. This is a compact encoding for this common function.

x86 Manual Vol. 1, page 3-22

X86手册：寻址模式的建议用法

Static arrays w/ fixed-size elements

- **(Index * Scale) + Displacement** — This address mode offers an efficient way to index into a static array when the element size is 2, 4, or 8 bytes. The displacement locates the beginning of the array, the index register holds the subscript of the desired array element, and the processor automatically converts the subscript into an index by applying the scaling factor.
- **Base + Index + Displacement** — Using two registers together supports either a two-dimensional array (the displacement holds the address of the beginning of the array) or one of several instances of an array of records (the displacement is an offset to a field within the record).
- **Base + (Index * Scale) + Displacement** — Using all the addressing components together allows efficient indexing of a two-dimensional array when the elements of the array are 2, 4, or 8 bytes in size.

2D arrays

x86 Manual Vol. 1, page 3-22

其他 ISA 的权衡

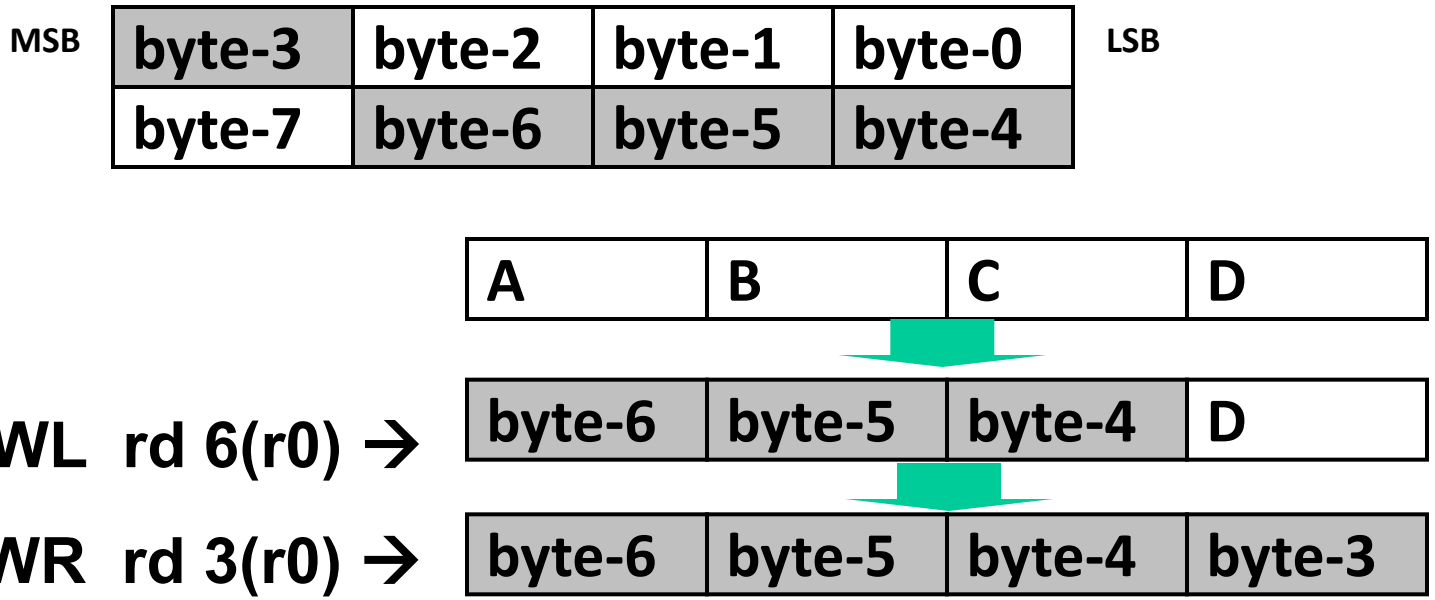
- 条件码 vs. 无条件码
- VLIW（超长指令字） vs. 单指令
- 精确异常 vs. 非精确异常
- 虚拟内存 vs. 无虚拟内存
- 未对齐访问 vs. 对齐访问
- 硬件互锁 vs. 软件保证互锁
- 软件管理 vs. 硬件管理页面错误处理
- 缓存一致性（硬件 vs. 软件）
- ……

程序员 vs. 微架构设计师

- **许多 ISA 特性旨在帮助程序员**
  - 这些特性能简化程序员的开发过程，但会增加硬件设计师的工作复杂度。
- **虚拟内存（Virtual Memory）**
  - 与覆盖编程（overlay programming）相比：
    - 虚拟内存使程序员无需关心物理内存的大小。
    - 然而，硬件设计者必须处理复杂的地址映射与内存管理问题。
  - 程序员是否需要关心代码块是否能适应物理内存？
    - 但硬件设计者需承担起映射与管理的责任。
- **寻址模式（Addressing Modes）**
  - 寻址模式决定了操作数如何获取，提供了多样化的选择，程序员可根据需要选择合适的模式来优化代码的效率。
- **未对齐的内存访问（Unaligned Memory Access）**
  - 程序员/编译器需要对齐数据：
    - 虽然未对齐访问可以增加灵活性，但它会增加硬件设计的复杂度，并可能影响性能。

MIPS：对齐访问（Aligned Access）

- **LW/SW 对齐限制：4 字节对齐（4-byte word-alignment）**
  - MIPS 体系结构要求每个字（word）必须按 4 字节对齐。
    - **不支持**从不对齐的内存字节中获取数据。
    - **不支持**将未对齐的字节旋转到寄存器中。
- **为“不常见”情况提供单独的操作码**
  - 对于访问不对齐数据的情况，MIPS 提供了专门的操作码：LWL（Load Word Left） 和 LWR（Load Word Right）。



X86：未对齐访问（Unaligned Access）

- **LD/ST 指令自动对齐数据**
  - LD（加载）和 ST（存储）指令会自动对齐跨越“字”边界的数据。
- **程序员/编译器无需关心数据存储位置**
  - 程序员或编译器不需要关心数据是否存储在对齐的位置（是否在字对齐位置）。

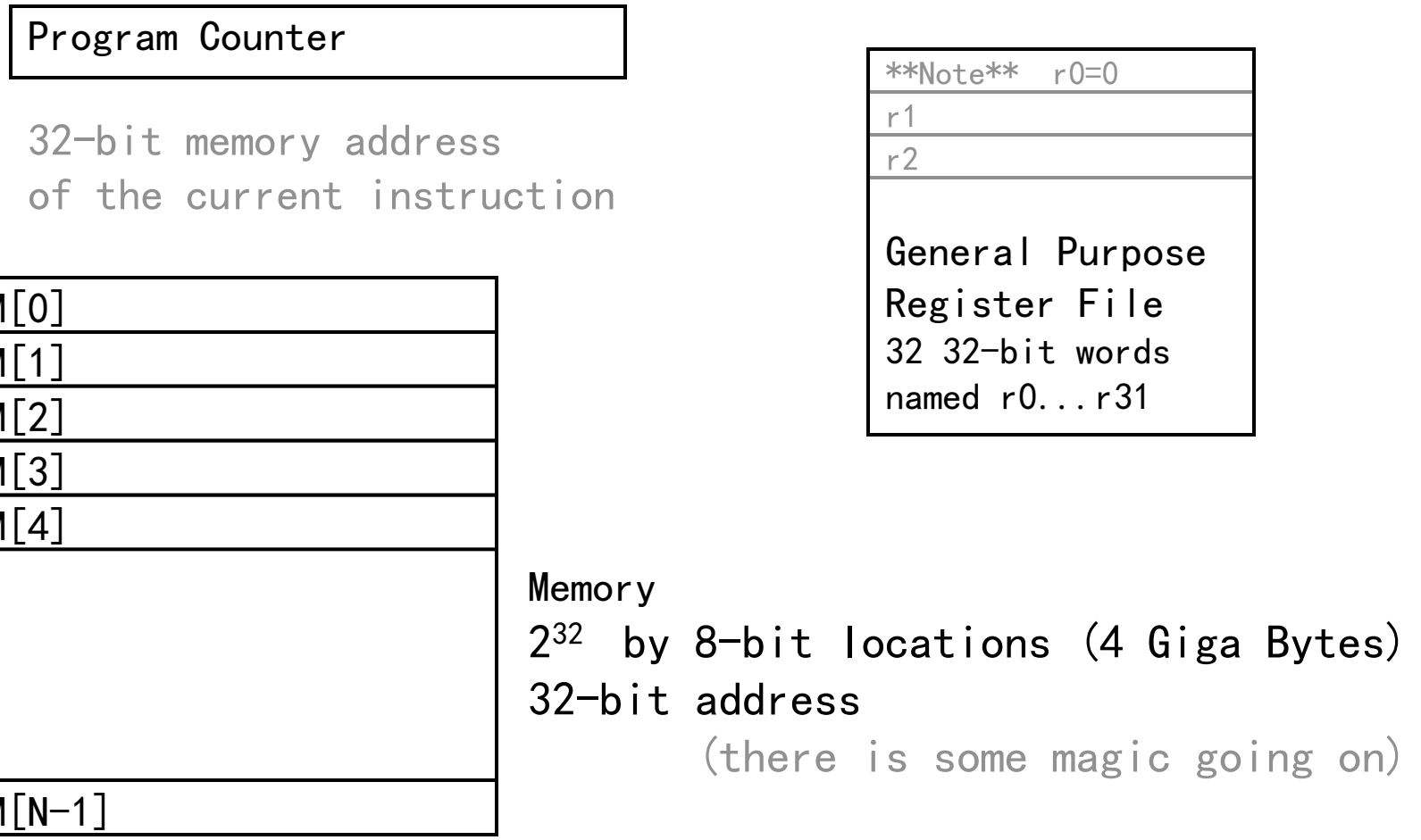
4.1.1 Alignment of Words, Doublewords, Quadwords, and Double Quadwords

Words, doublewords, and quadwords do not need to be aligned in memory on natural boundaries. The natural boundaries for words, double words, and quadwords are even-numbered addresses, addresses evenly divisible by four, and addresses evenly divisible by eight, respectively. However, to improve the performance of programs, data structures (especially stacks) should be aligned on natural boundaries whenever possible. The reason for this is that the processor requires two memory accesses to make an unaligned memory access; aligned accesses require only one memory access. A word or doubleword operand that crosses a 4-byte boundary or a quadword operand that crosses an 8-byte boundary is considered unaligned and requires two separate memory bus cycles for access.

今天的内容

- MIPS指令集简介

MIPS R2000 程序可见状态

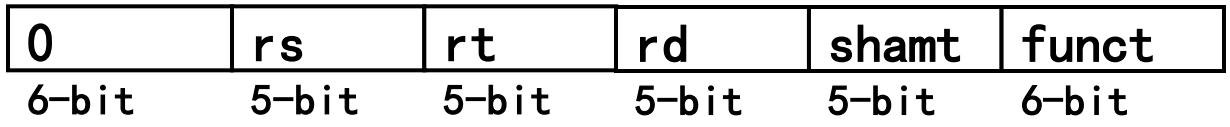


数据格式

- **大多数数据是32位：**
  - 指令和数据地址
  - 有符号和无符号整数
  - 只是位
- **还有16位字和8位字（即字节）**
- **浮点数：**
  - IEEE 754标准
  - 单精度：8位指数，23位尾数
  - 双精度：11位指数，52位尾数

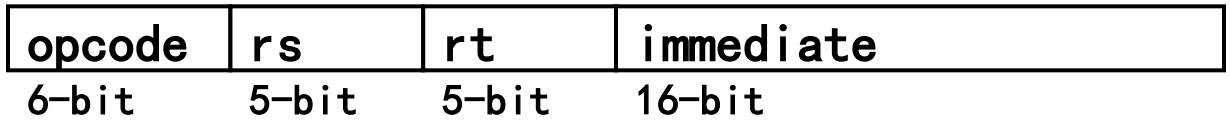
指令格式

- R 类型指令（R-type），3 个寄存器操作数



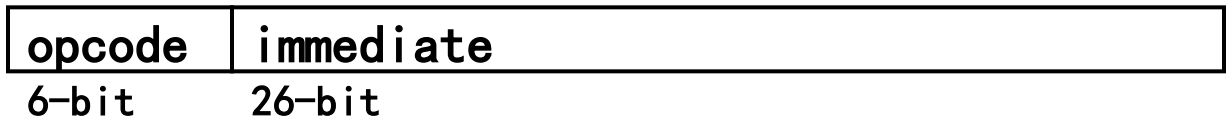
R-type

- I 类型指令（I-type），2 个寄存器操作数和 16 位立即数操作数



I-type

- J 类型指令（J-type），26 位立即数操作数

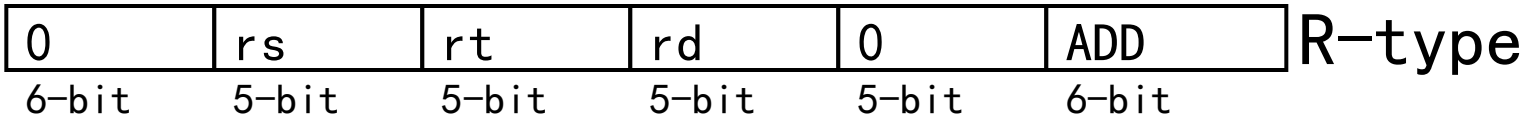


J-type

- 简易解码（Simple Decoding）
  - 每条指令长度为 4 字节，无论指令格式如何
  - 必须为 4 字节对齐（PC 的最低 2 位必须为 00）
  - 格式和字段容易在硬件中提取

ALU 指令

- 汇编语言（例如，寄存器-寄存器有符号加法）
  - ADD rd, rs, rt
- 机器编码



- 语义
  - GPR[rd] ← GPR[rs] + GPR[rt]
  - PC ← PC + 4
- 溢出异常
- 变种：
  - 算术运算：有符号、无符号 × {ADD, SUB}
  - 逻辑运算：{AND, OR, XOR, NOR}
  - 移位运算：{左移、右移-逻辑、右移-算术}

寄存器到寄存器（Reg-Reg）指令编码

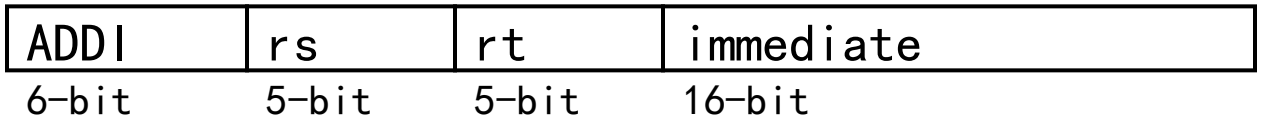
SPECIAL function

5...3 \ 2...0	0	1	2	3	4	5	6	7
0	SLL	*	SRL	SRA	SLLV	*	SRLV	SRAV
1	JR	JALR	*	*	SYSCALL	BREAK	*	SYNC
2	MFHI	MTHI	MFLO	MTLO	DSLLVε	*	DSRLVε	DSRAVε
3	MULT	MULTU	DIV	DIVU	DMULTε	DMULTUε	DDIVε	DDIVUε
4	ADD	ADDU	SUB	SUBU	AND	OR	XOR	NOR
5	*	*	SLT	SLTU	DADDε	DADDUε	DSUBε	DSUBUε
6	TGE	TGEU	TLT	TLTU	TEQ	*	TNE	*
7	DSLLε	*	DSRLε	DSRAε	DSLL32ε	*	DSRL32ε	DSRA32ε

[MIPS R4000 Microprocessor User’s Manua

ALU 指令

- 汇编语言（例如，寄存器-立即数有符号加法）
  ADDI rt, rs, immediate
- 机器编码：



I-type

- 语义
  - GPR[rt] ← GPR[rs] + sign-extend(immediate)
  - PC ← PC + 4
- 溢出异常
- 变种：
  - 算术：{有符号、无符号} × {ADD, SUB}
  - 逻辑：{AND, OR, XOR, LUI}

寄存器-立即数（Reg-Imm）指令编码

31...29 \ 28...26 (Opcode)	0	1	2	3	4	5	6	7
0	SPECIAL	REGIMM	J	JAL	BEQ	BNE	BLEZ	BGTZ
1	ADDI	ADDIU	SLTI	SLTIU	ANDI	ORI	XORI	LUI
2	COP0	COP1	COP2	*	BEQL	BNEL	BLEZL	BGTZL
3	DADDIε	DADDIUε	LDLε	LDRε	*	*	*	*
4	LB	LH	LWL	LW	LBU	LHU	LWR	LWUε
5	SB	SH	SWL	SW	SDLε	SDRε	SWR	CACHE δ
6	LL	LWC1	LWC2	*	LLDε	LDC1	LDC2	LDε
7	SC	SWC1	SWC2	*	SCDε	SDC1	SDC2	SDε

[MIPS R4000 Microprocessor User’s Manual]

汇编编程

- 将高层次程序构造分解为一系列基本操作

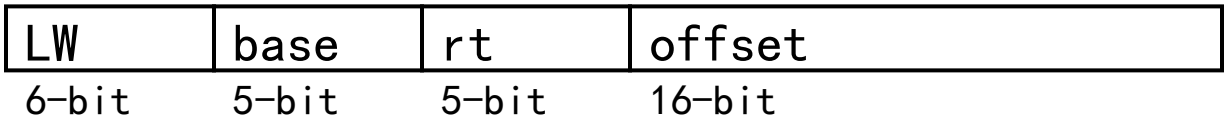
- 例如，高层次代码：
  - f = (g + h) - (i + j)

- 汇编代码：
  - 假设 f, g, h, i, j 在 r_f, r_g, r_h, r_i, r_j
  - 假设 r_{temp} 是空闲寄存器

```
add r_temp r_g r_h      # r_temp = g+h
add r_f r_i r_j  # r_f = i+j
sub r_f r_temp r_f      # f = r_temp - r_f
```

加载指令

- 汇编语言（例如，加载4字节单词）
  - LW rt, offset(base)
- 机器编码：

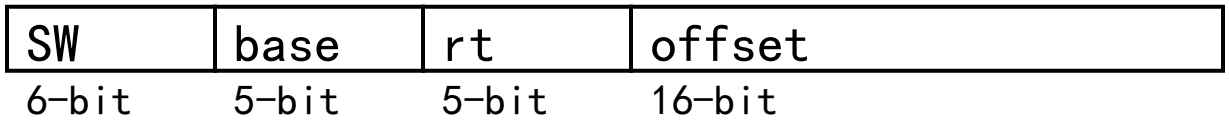


I-type

- 语义：
  - effective_address = sign-extend(offset) + GPR[base]
  - GPR[rt] ← MEM[translate(effective_address)]
  - PC ← PC + 4
- 异常：
  - 地址必须是“字对齐”的
  - MMU异常

存储指令

- 汇编语言（例如，存储4字节单词）
  - SW rt, offset(base)
- 机器编码：



I-type

- 语义：
  - effective_address = sign-extend(offset) + GPR[base]
  - MEM[translate(effective_address)] ← GPR[rt]
  - PC ← PC + 4
- 异常：
  - 地址必须是“字对齐”的
  - MMU异常

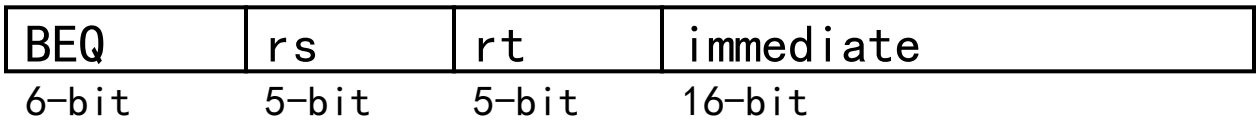
汇编编程

- 高阶代码示例：
  - A[8] = h + A[0]
  - 其中 A 是一个整数数组（每个元素4字节）
- 汇编代码：
  - 假设 &A，h 在 r_A，r_h
  - 假设 r_{temp} 是空闲寄存器

```
LW r_temp 0(r_A)        # r_temp = A[0]
add r_temp r_h r_temp   # r_temp = h + A[0]
SW r_temp 32(r_A)       # A[8] = r_temp
                        # note A[8] is 32 bytes
        #       from A[0]
```

（条件）分支指令

- 汇编语言（例如，如果相等则跳转）
  - BEQ rs, rt, immediate
- 机器编码：



I-type

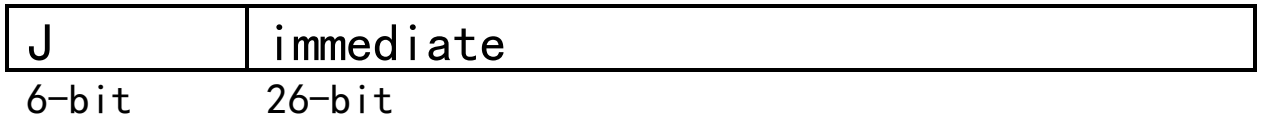
- 语义：
  - target = PC + sign-extend(immediate) * 4
  - 如果 GPR[rs] == GPR[rt]，则跳转到目标地址 PC ← target
  - 否则，PC ← PC + 4
- 跳转范围：
  - 每次跳转的距离由立即数决定，立即数符号扩展后乘以 4，来计算跳转的字节数。
- 变种：
  - BEQ, BNE, BLEZ, BGTZ

跳转指令

- 汇编语言（例如，无条件跳转）

  J immediate

- 机器编码：



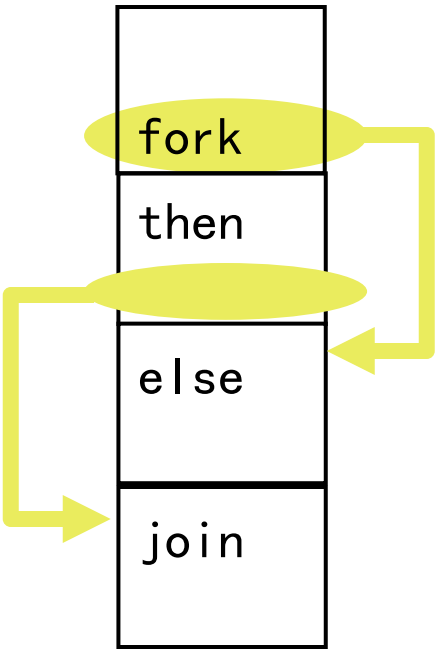
J-type

- 语义：
  - target = PC[31:28] << 28 | zero-extend(immediate) * 4
  - PC ← target
- 跳转范围：
  - 最大跳跃范围由立即数的26位决定，左移两位后表示28位，即最大跳跃范围为256MB。
- 变种：
  - 跳转与链接（Jump and Link, JAL）
  - 跳转寄存器（Jump Registers, JR）

汇编编程

- 高阶代码示例：

```
if (i == j) then
    e = g
else
    e = h
f = e
```



- 汇编代码：

假设 e, f, g, h, i, j 存储在寄存器 re, rf, rg, rh, ri, rj 中

```
bne ri, rj, L1        # L1 和 L2 是地址标签
add re, rg, r0        # e = g
j L2                  # 跳转到 L2
L1: add re, rh, r0    # e = h
L2: add rf, re, r0    # f = e
```

函数调用与返回

- **跳转与链接：JAL offset**
  - 返回地址 = PC + 8
  - 目标地址 = PC[31:28] << 28 | zero-extend(immediate) * 4
  - PC ← target
  - GPR[31] ← return address

- **在函数调用时，函数被调用者需要知道返回的位置。**

- **间接跳转：JR rs_reg**
  - 目标地址 = GPR[rs]
  - PC ← target

汇编编程

- Caller (调用者):

```
... code A ...
JAL _myfxn
... code C ...
JAL _myfxn
... code D ...
```

- Callee (被调用者):

```
_myfxn:     ... code B ...
            JR r31
```

MIPS R2000 寄存器使用约定

- r0：始终为0
- r1：为汇编器保留
- r2，r3：存储函数返回值
- r4~r7：存储函数调用参数
- r8~r15：“调用者保存的” 临时寄存器
- r16~r23：“被调用者保存的” 临时寄存器
- r24~r25：“调用者保存的” 临时寄存器
- r26，r27：为操作系统保留
- r28：全局指针
- r29：栈指针
- r30：“被调用者保存的” 临时寄存器
- r31：返回地址

总结：MIPS RISC

- **简单的操作**
  - 2输入，1输出的算术和逻辑操作
  - 少量的不同方式来完成相同的任务
- **简单的数据传输**
  - ALU 操作是寄存器到寄存器的（需要大量的寄存器文件）
  - “加载-存储”架构
- **简单的分支**
  - 分支条件和目标的种类有限
- **简单的指令编码**
  - 所有指令使用相同数量的位进行编码
  - 只有少数几种格式
- **简单来说，这是一个更适合编译器而非汇编程序员的指令集架构（ISA）**